EPDA 生命状态机设计文档（纯描述版·最终修订）

1. 概述

本 MOD 在 Garry's Mod 引擎原生生命系统的基础上，为玩家的存活状态扩展出三个濒死中间阶段：战斗、挣扎和抽搐。其核心机制是引入一个独立于引擎显示血量之外的“额外血量缓冲池”，在玩家受到致命伤害时，通过递推按份兑现的方式临时补充血量，从而延迟真正死亡，创造出富有层次感的濒死体验。

为了让这种濒死体验更加真实可信，系统不仅在服务端精确控制血量与状态，还会通过多层面的感官欺骗来强化濒死幻觉。当玩家进入挣扎或抽搐状态时，服务端会立即冻结玩家的所有操作输入，使其无法移动、攻击或互动；同时通知客户端强制切换到第三人称跟随视角，并隐藏第一人称的手臂和武器模型。这种“身体失控”与“视角剥离”的组合效应，使玩家在视觉和操作层面均产生“自己已经死亡”的强烈错觉，尽管角色实际上仍处于可干预的濒死状态。

每个玩家的生命周期都由一个专属的协程进行管理，该协程贯穿玩家的整个游戏过程，负责在不同阶段之间切换，并在恰当的时机执行冻结、解冻以及网络通知等操作。这种设计确保了状态流转的清晰可控，避免了复杂的事件嵌套。

系统完全尊重引擎的伤害结算权威，仅在必要时介入血量保护，并采用资源池预创建与幂等进入设计，确保逻辑清晰、性能稳定。

2. 引擎原生状态的扩展视角

在引擎内部，玩家仅有“存活”与“死亡”两种基本状态。存活状态下玩家可自由行动、攻击和交互；死亡后则生成布娃娃，等待重生。本 MOD 并不修改这一根本机制，而是在“存活”的大范畴内进一步细分出三个状态阶段。玩家的最终生死仍由引擎根据结算后血量是否归零来决定，MOD 只是在血量归零的边缘提供了一层可控的缓冲，并在表现层上用自己的布娃娃完全替代引擎默认尸体，同时配合客户端视角切换与操作冻结，营造出濒死即死的错觉。

3. 额外血量缓冲池与状态判定依据

系统为每个玩家维护一个名为“额外血量”的数值池，其初始总量等于挣扎血量与抽搐血量之和。该池独立于引擎管理的显示血量。当玩家受到伤害时，会优先扣除引擎显示血量；仅当显示血量不足以完全承受本次伤害时，系统才动用额外血量池进行递推保护。

额外血量池的当前剩余值直接决定了玩家所处的状态。当池子保持全满时，玩家处于战斗状态。当池子减少至仅剩抽搐血量份额时，玩家进入挣扎状态。当池子完全耗尽归零时，玩家进入抽搐状态。状态完全由这一数据定义，无需额外的历史记录或事件标记，保证了系统行为的高度可预测性。

4. 递推按份兑现的保命机制

面对一次足以致命的攻击，系统并不会一次性将全部额外血量转给玩家，而是采用“递推尝试、按份兑现”的策略。首先，系统尝试从额外血量池中取出一份挣扎血量加入引擎当前血量，并临时扩大血量上限以适应这部分增加。兑现后立即重新判断当前血量是否已足以承受本次伤害。如果足够，则兑现过程立即停止，伤害正常扣除，玩家存活并进入挣扎状态。

如果兑现第一份后仍不足以支付伤害，系统会继续尝试取出一份抽搐血量再次兑现。若这次兑现后血量达标，伤害照常结算，玩家进入抽搐状态。假如两份额外血量全部兑现后，玩家总血量依然低于伤害值，系统便不再

干预，玩家将真正死亡。整个兑现过程从不调用阻止伤害的函数，只是提前为玩家补充血量，随后完全交由引擎进行伤害扣除和生死裁决。

5. UI 最大血量的动态同步

为了向玩家准确传达当前濒死阶段的特征，UI 中显示的最大血量会随状态变化而动态调整。在战斗状态时，最大血量显示为玩家的基础生命上限。进入挣扎状态后，最大血量调整为挣扎血量的数值，表明玩家此时的血量上限已被压缩至较低水平。而在抽搐状态下，最大血量进一步降低至抽搐血量的数值。每次伤害结算后，系统都会根据剩余额外血量的情况刷新这一显示值，确保玩家看到的血条与自身处境完全相符。

6. 挣扎状态内部的动画驱动子状态

挣扎状态并非静止不变，其内部包含两个由动画文件本身驱动的子状态：濒死动画与爬行。当玩家首次进入挣扎状态时，系统会激活预先创建好的布娃娃和动画实体，将其对齐到玩家当前位置，并播放一段倒地挣扎的死亡动画。该动画的持续时间由动画文件自身决定，系统通过读取其精确时长来设定内部计时器。

一旦动画播放完毕，子状态会自动切换至爬行阶段。此时动画源会切换为爬行循环，玩家将能够以极慢的速度移动，模拟濒死爬行求生。两个子状态的切换完全依赖于动画时长，无需任何硬编码的定时器，从而实现了视觉表现与游戏逻辑的精确同步。

7. 资源池预创建与幂等进入设计

为避免在战斗过程中频繁创建和销毁实体导致性能波动，系统采用资源池预创建策略。每个玩家在首次进入游戏时，就会提前生成所需的辅助实体：用于表现倒地姿态的物理布娃娃、用于驱动自定义动画的动画源，以及一个替代玩家与世界进行碰撞的代理球体。这些实体在战斗状态下保持隐藏和无碰撞，仅在玩家进入挣扎或抽搐状态时才会被激活。

此外，所有状态进入动作均被设计为幂等操作。无论因何种原因、在何时调用状态进入函数，只要玩家已处于该状态且对应资源已经激活，函数就不会重复创建实体或重播动画。这一设计极大地简化了状态同步逻辑，使得系统在每次伤害事件后都可以放心地执行一次状态对齐，而无需关心事件的起因或历史路径。

8. 客户端视角欺骗与操作冻结

为了让玩家在濒死时获得更强烈的心理冲击，系统会在服务端状态切换的同时，向客户端发送网络消息，并立即冻结玩家的所有操作输入。玩家将无法移动、跳跃、攻击或使用物品，彻底丧失对身体的掌控感，进一步强化“我已无力反抗”的绝望氛围。

与此同时，客户端在收到进入干预状态的消息后，会执行两项视觉欺骗操作：第一，强制隐藏第一人称的视角模型，使玩家无法再看到自己手中的武器和手臂；第二，接管视角计算，将摄像机从玩家眼部位置拉远至一个固定的第三人称跟随视角，通常位于玩家后上方，模拟死后旁观自己尸体的观感。该视角会持续注视玩家角色的身体，并根据玩家位置动态调整，同时会进行简单的视线追踪以防止摄像机穿墙。

只要干预状态未退出，客户端便会始终保持这种第三人称旁观视角，并维持操作冻结，即使玩家实际上仍在挣扎或爬行。当玩家最终死亡或重生返回战斗状态时，服务端才会解冻操作，并通知客户端退出干预状态，恢复正常的第一人称视角和武器模型显示。这种身体失控与视角剥离的双重欺骗手段，使得濒死体验在感官层面几乎与真实死亡无异，极大地增强了系统的表现力和沉浸感。

9. 死亡处理与布娃娃接管

当玩家因额外血量耗尽而真正死亡时，引擎原本会自动生成一个标准的布娃娃作为尸体。本 MOD 会主动移除该引擎布娃娃，并始终使用自己预创建的物理布娃娃作为玩家角色的最终尸体表现。该布娃娃在玩家进入挣扎或抽搐状态时便已激活并跟随动画，因此能够无缝延续死亡瞬间的姿态和位置，提供更具沉浸感和一致性的视觉体验。玩家自始至终都将系统创建的布娃娃视为自己的化身实体，无论处于濒死挣扎还是彻底死亡。而客户端早在进入干预状态时便已切换至旁观视角，因此死亡时刻的视觉过渡也显得极为自然。

10. 重生时的状态重置

当玩家按下鼠标左键重生时，系统会将一切重置回初始状态：额外血量池重新填满至全量，状态恢复为战斗，所有预创建的资源实体重新隐藏并回到待命状态，玩家的原始模型、移动能力和碰撞属性也完全复原。同时，服务端会解除操作冻结，并通知客户端退出干预状态，恢复第一人称视角和武器模型的显示。这一过程保证了每次重生都是一个全新且干净的周期，为下一次濒死体验做好准备。

11. 玩家生命周期的协程管理

每个玩家的濒死状态流转并非由零散的事件钩子拼凑而成，而是由一个贯穿其整个游戏生涯的协程统一管理。该协程在玩家首次生成时启动，并持续运行直至玩家断开连接。协程内部是一个清晰的状态机循环，它根据玩家当前的存活状态和濒死阶段，依次执行不同分支的进入、更新和退出逻辑。

在战斗状态下，协程几乎不进行任何干预，玩家操作完全自由。一旦状态从战斗切换至挣扎或抽搐，协程会立即执行冻结操作并通知客户端进入干预模式，随后根据具体的子状态调用对应的进入函数（如激活资源实体、播放动画），并在每一帧通过更新函数维持子状态内的逻辑（如检查动画是否结束）。当状态因血量变化或死亡而发生改变时，协程会先执行当前状态的退出清理，再转入下一状态的循环。这种集中式的流程控制使得状态的进入、退出边界极为清晰，避免了因钩子触发顺序不确定性而导致的状态错乱。

12. 伤害处理流程中的钩子协作

尽管协程负责状态流转，但状态何时切换的决策信息来源于伤害事件。系统依赖于服务端的两个核心钩子协同工作。第一个钩子在伤害即将扣除前触发，它负责评估引擎血量是否足够，并在必要时执行递推兑现操作，临时提升玩家的最大血量和当前血量。此阶段不记录任何临时变量，仅完成血量补充。

第二个钩子在玩家实际受到伤害、血量已经扣除后触发。由于该钩子直接提供了剩余血量等丰富信息，系统便在此处更新上下文中的当前状态标识。协程在下一帧的更新循环中检测到状态标识的变化后，便会自动执行相应的状态切换流程。这种“钩子采集数据并更新标识，协程响应标识并驱动表现”的分工模式，使得兑现逻辑与表现逻辑彻底解耦，代码职责清晰且不易出错。

13. 设计原则总结

本系统的设计自始至终遵循几条简明原则。第一，状态完全由剩余额外血量这一单一数据源决定，消除了复杂的事件记忆。第二，兑现过程递推且按固定份额进行，既避免过度兑现，又确保状态切换边界明确。第三，引擎的伤害结算始终保持原样，系统仅通过临时输血来保护玩家，不破坏引擎内部逻辑。第四，所有表现性操作均具备幂等性，允许状态同步在任何时刻安全执行。第五，资源池预创建保障了性能稳定，避免了运行时创建开销。第六，布娃娃的完全接管保证了从濒死到死亡视觉表现的一致性。第七，操作冻结与客户端视角欺骗共同构成多感官的濒死幻觉，极大地增强了沉浸感。第八，协程统一管理玩家生命周期，使状态流转清晰可控。这些原则共同构成了一个健壮、可维护且扩展友好的濒死状态系统。